WEIGHTED DOWNSET CACHING FOR DETERMINISTIC 3SUM

JIONGHAN WANG

Abstract. We refine the ternary-word algorithm for deterministic integer 3SUM by choosing its cache according to an additive digit cost. The cache uses weights $(0, 512, 1025)$, while homogeneous reconstruction retains the usual degree $(0, 1, 2)$. We prove exact reconstruction from the weighted cache and bound both the uncached states and the complete inverse-query tree by generating functions. After retuning the type distribution and charging the full solver, we obtain a deterministic word-RAM running time of $O(n^{96013971/50000000}) = O(n^{1.92027942})$ for polynomially bounded integer inputs. This lowers the exponent of the inherited 1.9202794287989125324 certificate by approximately $8.8 \cdot 10^{-9}$. The proof is formalized in Lean, from the physical cache procedures to the original distinct-position decision problem and its word-RAM semantics.

1. Introduction

Integer 3SUM asks whether an array $x_1, \ldots, x_n$ contains distinct positions i, j, k with $x_i + x_j + x_k = 0$. Alman and Vassilevska Williams established a deterministic truly subquadratic algorithm for polynomially bounded integers through sparse lopsided triangle problems [1]. Subsequent formal developments use ternary encodings, binomial transforms, and deterministically selected hash rows to obtain a substantially smaller exponent [4]. We refine the cache geometry inside this ternary construction.

The central observation is that two additive quantities can serve different purposes. Ordinary degree controls an exact Euler reconstruction identity. A separate monotone digit cost controls which transform values are stored. This separation permits a slightly asymmetric cache, whose storage and query exponents improve after retuning the surrounding type distribution.

Theorem 1.1 (Deterministic integer 3SUM)**.** *For every fixed $\kappa \in \mathbb{N}$, there is a finite deterministic word-RAM program that decides 3SUM on n integers of absolute value at most n^κ in*

$$O(n^\tau) \quad \textit{time}, \qquad \tau = \frac{96013971}{50000000} = 1.92027942.$$

The program works at every input size and every word width $W \geq b(\lfloor \log_2 n \rfloor + 1)$, for a constant b depending only on κ. Equal values at different positions are permitted.

The RAM has signed addition, subtraction, multiplication, indirect loads and stores, and conditional branches, each costing one step. Its arithmetic and execution semantics are the inherited `EndStatement` specification [4]. The result concerns the decision problem.
Contribution and context. Our contribution consists of the weighted cache, its reconstruction and query analysis, an exact parameter certificate, and their integration into the complete verified solver. We build on the ternary construction of c0ldzy17 and contributors, including Joshua Wharton's numerical certificates and Hrishik Sai Bojnal's parameter refinements [4]. The comparison exponent is the exact rational 4800698571997281331/2500000000000000000 of the immutable source snapshot used here; the benchmark records its provenance [2]. The broader transform approach is related to fast subset convolution [3]. Our improvement is an

Date: October 31, 2026.
Key words and phrases. 3SUM, deterministic algorithms, word RAM, binomial transforms, weighted downsets, formal verification.

asymptotic refinement of this existing construction. The fixed parameters are large, so the result is most naturally viewed as a theorem about the attainable exponent.

We first establish the algebraic kernel, then analyze the weighted cache and its executable queries. The global cost ledger translates these bounds into the exponent in Theorem 1.1. The final section explains the complete formal certificate and its reproduction.

2. The ternary transform kernel

Let $\mathcal{W}_N = \{0,1,2\}^N$, ordered coordinatewise. Write

$$\rho(R) = \sum_{i=1}^{N} R_i, \qquad \binom{R}{A} = \prod_{i=1}^{N} \binom{R_i}{A_i}, \qquad R! = \prod_{i=1}^{N} R_i!.$$

For $F : \mathcal{W}_N \to \mathbb{Z}$, define the binomial transform and its inverse by

$$(\mathcal{Z}F)(R) = \sum_{A \le R} \binom{R}{A} F(A), \tag{2.1}$$

$$(\mathcal{D}H)(Q) = \sum_{R \le Q} (-1)^{\rho(Q)-\rho(R)} \binom{Q}{R} H(R). \tag{2.2}$$

These are tensor products of the three-point binomial transform and its inverse. A function F is *homogeneous of degree h* when $F(A) \neq 0$ implies $\rho(A) = h$.

Lemma 2.1 (Euler reconstruction)**.** *If F is homogeneous of degree h, then*

$$(\rho(R) - h)(\mathcal{Z}F)(R) = \sum_{i : R_i > 0} R_i (\mathcal{Z}F)(R - e_i). \tag{2.3}$$

Consequently, the values of $\mathcal{Z}F$ on any set containing every word of rank at most h determine all its values uniquely.

Proof. For every i and $A \le R$, the binomial identity

$$R_i \binom{R - e_i}{A} = (R_i - A_i) \binom{R}{A}$$

holds, with the zero-coordinate term interpreted as zero. Multiply by $F(A)$ and sum over A and i. Homogeneity makes $\sum_i (R_i - A_i) = \rho(R) - h$ on the support of F, proving (2.3). At any remaining word, $\rho(R) > h$. Division by this positive integer reconstructs its value from strictly lower ranks. Induction on rank proves uniqueness and exact integer reconstruction. □

The transform computes the ordinary additive convolution once the inputs have been scaled by factorials. Put

$$(f * g)(Q) = \sum_{\substack{A,B \in \mathcal{W}_N \\ A+B=Q}} f(A) g(B), \qquad \widehat{f}(A) = A! f(A), \quad \widehat{g}(B) = B! g(B),$$

where $A + B = Q$ is an equality of integer vectors.

Lemma 2.2 (Top-degree convolution)**.** *Suppose that f and g have degrees a and b, respectively. If $\rho(Q) = a + b$, then*

$$\mathcal{D}\big((\mathcal{Z}\widehat{f})(\mathcal{Z}\widehat{g})\big)(Q) = Q!\,(f * g)(Q). \tag{2.4}$$

Proof. Expand the left-hand side and factor its coefficient for $f(A)g(B)$ coordinatewise. The one-coordinate coefficient is

$$\sum_{r=0}^{q} (-1)^{q-r} \binom{q}{r} \binom{r}{a} \binom{r}{b}.$$

It vanishes when $a + b < q$, by finite differences of a polynomial of degree at most $a + b$. If $a + b = q$, its value is $q!/(a!b!)$. For a nonzero global contribution, each coordinate must

satisfy $A_i + B_i \geq Q_i$. Homogeneity and $\rho(Q) = a + b$ force equality in every coordinate. The factorials in $\widehat{f}, \widehat{g}$ then leave exactly $Q!$. □

Thus a query is answered by two homogeneous reconstructions, a pointwise product, an inverse query, and exact division by $Q! = 2^{|\{i:Q_i=2\}|}$.

3. Weighted downset caching

3.1. **The cache and its reconstruction domain.** Assign the digit costs

$$c(0) = 0, \qquad c(1) = 512, \qquad c(2) = 1025, \qquad C(R) = \sum_i c(R_i). \tag{3.1}$$

For an integer threshold $T \geq 0$, define

$$\mathcal{B}_T = \{R \in \mathcal{W}_N : C(R) \leq T\}.$$

The monotonicity of c makes $\mathcal{B}_T$ a downset. Relative to the proportional costs $(0, 512, 1024)$, the extra unit on digit 2 distinguishes words of equal ordinary degree. Retuning the threshold and output type balances the resulting storage and query charges. For every R,

$$2C(R) \leq 1025\rho(R). \tag{3.2}$$

Hence $1025h \leq 2T$ guarantees that $\mathcal{B}_T$ contains every word of rank at most h, as required by Lemma 2.1.

For $R \leq Q$, define its cost deficit $\Delta(Q, R) = C(Q) - C(R)$ and the near set

$$\mathcal{U}(Q, D) = \{R \leq Q : \Delta(Q, R) \leq D\}.$$

When $C(Q) > T$, the uncached words below Q are exactly

$$\{R \leq Q : C(R) > T\} = \mathcal{U}(Q, C(Q) - T - 1). \tag{3.3}$$

Every predecessor needed by Euler reconstruction is either in this near set or in the bank. Evaluating the near set in increasing ordinary rank therefore computes all required values once.

Lemma 3.1 (Weighted counting bounds)**.** *Let $n_d(Q) = |\{i : Q_i = d\}|$. For $0 < y, z < 1$,*

$$|\mathcal{B}_T| \leq \frac{(1 + y^{512} + y^{1025})^N}{y^T}, \tag{3.4}$$

$$|\mathcal{U}(Q, D)| \leq \frac{(1 + z^{512})^{n_1(Q)}(1 + z^{513} + z^{1025})^{n_2(Q)}}{z^D}. \tag{3.5}$$

Proof. For $R \in \mathcal{B}_T$, $y^T \leq y^{C(R)}$. Sum this inequality over the bank and extend the sum to all words. The resulting generating function factors into $(1 + y^{512} + y^{1025})^N$. Similarly, $z^D \leq z^{\Delta(Q,R)}$ on $\mathcal{U}(Q, D)$. The possible coordinate deficits are $\{0\}$, $\{0, 512\}$, and $\{0, 513, 1025\}$ for query digits $0, 1, 2$. Factoring their generating functions proves (3.5). □

3.2. **Complete inverse queries.** For $H = (\mathcal{Z}\widehat{f})(\mathcal{Z}\widehat{g})$, let K_j be the result of applying the inverse transform in the first j coordinates. The bank stores $K_j(R)$ for $0 \leq j \leq N$ and $R \in \mathcal{B}_T$. All these values can be computed inside the downset.

To evaluate $K_N(Q)$, recursively apply the inverse transform in a fixed coordinate order. At a node (j, R) with $R \in \mathcal{B}_T$, return the stored value $K_j(R)$. Other nodes recurse on the allowed values of the current digit. At stage zero, the value is the reconstructed product $H(R)$. This procedure follows the same coordinate recurrence as the full inverse transform and consequently returns (2.2).

Lemma 3.2 (First-crossing bound)**.** *For an uncached query Q, the number of nodes in the complete inverse-query tree is at most*

$$(N + 1)\,|\mathcal{U}(Q, C(Q) + 1024 - T)|. \tag{3.6}$$

Proof. Before the first entry into the bank, a node has cost at least $T+1$. Changing one digit decreases cost by at most 1025. The first bank node therefore has cost at least $T-1024$. Both kinds of node have deficit at most $C(Q)+1024-T$. At each depth, the processed coordinates and their assigned digits uniquely identify the node's word, while unprocessed coordinates retain their digits from Q. Thus each depth injects into this near set. Summing over at most $N+1$ depths proves the bound. □

Combining Lemmas 3.1 and 3.2 charges the entire tree, including its first cached leaves. In particular, its generating-function bound contains the fixed factor z^{-1024}.

3.3. **Executable tables and queries.** The implementation stores words in increasing address order and uses binary search to locate compact table entries. A word has the two-bit-per-coordinate address $\sum_{i=0}^{N-1} R_{i+1}4^i$. Depth-first enumeration prunes a partial word once its cost exceeds the remaining budget. Monotonicity makes this enumeration complete; the induced address order gives sorting and uniqueness. The same procedure, with deficits as budgets, enumerates near sets.

For each pair of homogeneous source arrays, the query kernel performs:

(1) bank enumeration, source initialization, and forward transforms;
(2) pointwise multiplication and storage of all inverse bank stages;
(3) near-set enumeration and Euler reconstruction for each source;
(4) multiplication of the reconstructed values, the complete frozen inverse query, and factorial division.

Bank construction is shared by all queries using the same source pair.

Let $b=|\mathcal{B}_T|$ and $r=|\mathcal{U}(Q,C(Q)-T-1)|$. With address-search parameter L_{addr} and a proved arithmetic magnitude bound V, write $\Lambda=N+L_{\mathrm{addr}}+\lfloor\log_2(V+1)\rfloor+4$. The checked finite bound for bank enumeration, three bank sweeps, two memoized reconstructions, near enumeration, and the corresponding products is

$$35000\,\Lambda^3(b+r+1). \tag{3.7}$$

The implemented exact-division routines are charged through the logarithmic dependence on V. Initialization and the complete inverse query have separate checked bounds and are included when the caller combines these procedures. Thus preprocessing and each query cost a polynomial factor times their respective bank and near-set bounds. The magnitude and address bounds also establish a polynomial space and word envelope for the concrete procedures.

4. From generating functions to a time exponent

4.1. **Types and principal dimensions.** Let $\pi=(\pi_0,\pi_1,\pi_2)$ be the output type: an output block of length ℓ has $\pi_d\ell$ digits equal to d. The two source types are both

$$\alpha_0=\pi_0+\pi_1/2+\pi_2/3,\quad \alpha_1=\pi_1/2+\pi_2/3,\quad \alpha_2=\pi_2/3. \tag{4.1}$$

This is the balanced marginal of the two decompositions of digit 1 and the three decompositions of digit 2. In particular, output degree is twice source degree.

All logarithms in this section are natural. Define

$$H=\mathcal{H}(\pi)=-\sum_d \pi_d\log\pi_d,\qquad A=\mathcal{H}(\alpha), \tag{4.2}$$

$$G=\pi_1\log 2+\pi_2\log 3,$$

$$B=\log(1+y^{512}+y^{1025})-t\log y, \tag{4.3}$$

$$Q_*=\pi_1\log(1+z^{512})+\pi_2\log(1+z^{513}+z^{1025})-d\log z, \tag{4.4}$$

where $t=T/N$ and $d=C(Q)/N-t$. The notation Q_* denotes a rate, while Q denotes a query word. The admissibility conditions include $1025(\alpha_1+2\alpha_2)<2t$, so the cache covers

TABLE 1. Principal exponents and certified upper bounds. All displayed bounds are rounded upwards from exact rational enclosures.

Operation family	Limiting exponent	Upper bound
Cache tables	$2B/H$	1.920279410509
Requested convolution queries	$2 - G/H + 2Q_*/H$	1.920279410538
Source dictionaries	$2A/H$	1.255583372078
Residue histograms	$1 + G/H$	1.251789128452
Requested incidences	$2 - G/H$	1.748210871549
Linear work	1	1.000000000000

the source degree layer. For words of the prescribed output type, Lemmas 3.1 and 3.2 give

$$|\mathcal{B}_T| \le e^{NB}, \qquad \text{inverse-query nodes} \le (N+1)z^{-1024}e^{NQ_*}$$

We recall how the inherited encoding turns these local rates into global charges. Let M_ℓ and A_ℓ be the output and source type-class sizes, and put $R_\ell = 2^{\pi_1 \ell} 3^{\pi_2 \ell}$. Multinomial estimates give

$$\log M_\ell = \ell H + O(\log \ell), \quad \log A_\ell = \ell A + O(\log \ell), \quad \log R_\ell = \ell G.$$

The finite prime construction chooses a prime with limiting rate $\ell^{-1} \log p_\ell = (H+G)/2$, up to a separately charged inflation that tends to zero as its fixed control parameter grows. The number of blocks is $s = \lceil \log_{M_\ell}(n^2) \rceil$.

The inherited framework represents each colored equation $a + b = c$ by local source words of type α and an output word of type π. Local covers and signed carries compose these representations across blocks. At each prefix, the source histograms are grouped by their current hash keys, and the requested output words specify the residue equations to be counted. The deterministic selector evaluates candidate hash rows through these exact counts and their weighted moment channels. Thus the interface required of the replacement kernel is exact typed convolution, together with its physical resource and time bounds.

At prefix $j \le s$, the principal dimensions, with representation-family factors charged separately, are the bank $e^{j\ell B}$, source tables A_ℓ^j, histogram domain p_ℓ^j, and the requested-incidence scale $n(M_\ell/p_\ell)^j$. The latter each incur a query factor $e^{j\ell Q_*}$. Taking logarithms, using $s = 2\log n/\log M_\ell + O(1)$, and then increasing the fixed block size yields the six charges in Table 1. Finite prime inflation, representation families, carries, and the number of prefixes are charged in the complete assembly below.

4.2. **An exact certificate.** We take the following terminating rational output distribution:

$$\pi = (0.8120584037,\ 0.1586669416,\ 0.0292746547).$$

The source distribution is defined by (4.1), and

$$t = \frac{130448080617511}{1250000000000}, \qquad d = \frac{134483020951}{19531250000}, \tag{4.5}$$

$$y = \frac{199334926009}{200000000000}, \qquad z = \frac{497480529693}{500000000000}. \tag{4.6}$$

These satisfy $t + d = 512\pi_1 + 1025\pi_2$ and the strict base-layer condition. The integer schedule in Appendix A realizes all counts and thresholds exactly. The profile also satisfies the inherited cover conditions, including

$$G < H, \qquad (H+G)/2 < A < B, \qquad 2\pi_1 + 3\pi_2 < 1.$$

For the mixed-projection fibers, binary entropy $h_{\mathrm{bin}}(u) = -u\log u - (1-u)\log(1-u)$ satisfies $h_{\mathrm{bin}}(\alpha_2) < (\log 2)/4 < H/2$. These inequalities ensure that the retuned source and output types admit the same finite-field encoding and local-cover construction.

Proposition 4.1 (Principal certificate)**.** *For these parameters, all six principal exponents in Table 1 are strictly smaller than*

$$\gamma_0 = \frac{480069853}{250000000} = 1.920279412 < \tau, \qquad \tau - \gamma_0 = \frac{1}{125000000}.$$

Proof. The exact rational logarithm enclosures in the accompanying `Parameters.lean` imply bounds on H, A, G, B, Q_*. Since $H > 0$, each quotient comparison reduces to a linear inequality. For example, the bank and query comparisons are

$$2B < \gamma_0 H, \qquad 2H - G + 2Q_* < \gamma_0 H.$$

The remaining comparisons are $2A < \gamma_0 H$, $H + G < \gamma_0 H$, $2H - G < \gamma_0 H$, and $1 < \gamma_0$. Lean verifies these inequalities by exact rational arithmetic. For the logarithms, range reduction is followed by the finite series

$$\log \frac{1+u}{1-u} = 2 \sum_{j=0}^{m-1} \frac{u^{2j+1}}{2j+1} + \mathcal{R}_m, \qquad |\mathcal{R}_m| \le \frac{2|u|^{2m+1}}{(2m+1)(1-|u|^2)}.$$

The positive generating polynomials are first enclosed rationally, so their logarithms use the same certified argument. □

4.3. **Complete overhead and parameter selection.** The inherited global solver [4] computes its finite field, local dictionaries, and row menu; executes the selected-prefix loop; and performs collision cleanup and final detection. Our modification supplies exact convolution values to these callers by Lemma 2.2. The equality is an integer equality, so it also preserves the weighted moment channels used by the selector.

The complete analysis uses the strict chain

$$1.920279412 < 1.9202794152 < 1.9202794168 < 1.92027942. \tag{4.7}$$

The first gap pays finite block rates, representation multiplicity, and deterministic moment costs. The next gap accommodates the signed digit carry envelope and the remaining whole-program budget. The last gap converts this budget to the stated RAM exponent.

Proposition 4.2 (Complete execution budget)**.** *For each fixed κ, the modified construction admits fixed parameters and an executable fast solver, correct above a fixed cutoff, with polynomial resource requirements and a size-only time budget T_{fast} satisfying*

$$\text{for every } \varepsilon > 0, \quad T_{\mathrm{fast}}(n) \le n^{1.9202794168+\varepsilon} \quad \text{for all sufficiently large } n. \tag{4.8}$$

Proof. Fix the field-inflation parameter $K \ge 10^{20}$ and the inherited moment order $k = 30000000000000K^2$. For fixed K, the finite type-class rates converge as the block size grows. Inflation and moment-rate allowances are bounded by terms of order $1/K$ and $1/k$, respectively, within the first gap in (4.7). Proposition 4.1 supplies strict room for the finite prime and type-class rates. The local cover, weighted histogram, tensor, and reporting budgets are charged to these finite dimensions.

Next choose a signed-digit radix $\beta = 2^r$ and a sufficiently large integral block. The simultaneous parameter theorem ensures the field-minor conditions, the representation bound, and a carry exponent below $1/(100K^2)$. This choice depends only on κ. The prefix loop reduces its ordered collision score by $pS_{j+1} \le 9S_j$, from $S_0 \le n^2$. The selected parameters give a final collision count at most $n^{0.7483}$, within the charged cleanup budget.

Each source histogram is constructed from stored input records. For every prefix, the exact type gives the cache cost, and the physical bank and near-set enumerations supply every required table entry. Equation (3.7), the initialization bounds, and Lemma 3.2 price the full query, including z^{-1024}. The global budget takes a finite maximum over all typed query times and then charges its use in every actual call. It includes the complete row search, weighted moments, replay, cleanup, and final detector.

These estimates yield the size-only envelope at the third exponent in (4.7); polynomial digit and prefix factors fit inside the arbitrary ε in (4.8). The simultaneous parameter choice

and complete execution budget are formalized in the profile and runtime modules listed in Appendix B. □

5. The complete certificate

5.1. **Original inputs and machine execution.** The outer deterministic coloring family separates every triple of distinct original positions in some coloring. Each colored call uses the exact additive convolution above; source and requester records retain their occurrence indices and representation multiplicities. The inherited selected-row and cleanup proofs therefore apply to the new kernel's exact counts. At termination the detector's verdict is equivalent to the original distinct-position zero-sum predicate.

The formal implementation uses the inherited *Light* program language, whose operational contracts specify return values, memory contents, preserved regions, time, word bounds, and call depth. We instantiate these contracts with concrete procedure lists: a 38-procedure cache library, a 64-procedure count frontend, and a 135-procedure source/count host. The global execution proof establishes their premises from previously constructed physical memory. In particular, the header computes the output cost as

$$(1024\,h_0 + 3a_2)s,$$

where h_0 and a_2 are a local source's ordinary rank and number of twos, and s is the prefix length. The exact type proves that this equals the query's weighted cost.

Proof of Theorem 1.1. Instantiate Proposition 4.2 for κ. The concrete execution theorem proves the original decision predicate and the polynomial word, memory, and stack requirements. A hybrid wrapper uses the verified triple scan on the finite exceptional input sizes. It therefore solves all inputs in the specified magnitude class.

Choose $\varepsilon > 0$ with $1.9202794168 + \varepsilon < \tau$. The strict margin absorbs the compiler's fixed overhead and yields a time function $T(n) = O(n^\tau)$. The inherited verified compiler transfers the Light contract to the signed word-RAM semantics for all admissible word widths. The final theorem quantifies over κ, giving precisely the statement of Theorem 1.1. □

5.2. **Verification and reproduction.** The public Lean statement is

```
theorem WeightedRace.bound :
  EndStatement.ThreeSum.SolvedInTime
    (96013971 / 50000000 : Rat)
```

Here the exponent is an exact rational. The specification expresses its Big-O condition as $T(n)^{50000000} \le C_0 n^{96013971}$ for all $n \ge 2$, for a suitable integer constant C_0.

The development uses Lean 4.33.1 [5] and Mathlib [7], pinned to revision `0df444a3`; the artifact records the full revision identifier. The full final dependency graph builds successfully. A subsequent build of all added modules completed 4542 build tasks. The combined verification log records 57 axiom reports, including ten dedicated final-theorem and whole-solver checks. Their dependencies are contained in {`propext`, `Classical.choice`, `Quot.sound`}. The exact problem specification and all 1201 inherited Lean source files match the immutable upstream snapshot byte for byte.

Artifact availability. The accompanying formal artifact contains the full Lean sources, pinned dependency metadata, the final audit, and verification receipts. Its README gives the build commands. The manuscript's source archive includes this artifact as an ancillary file. The key commands, from the artifact root, are:

```
lake build Submission.Weighted
lake env lean AuditWeightedSolver.lean
```

Appendix B maps the main arguments to the corresponding source files.

Acknowledgments and tool use. The construction and formalization build on the work credited in [1, 4], including Anthropic's word-RAM specification, program logic, and verified compiler. The upstream development also credits Swapnil Jain's deterministic filtering work [6]. OpenAI Codex assisted with algorithm exploration, Lean development, and manuscript preparation. Lean checks the formal proof terms; the companion artifact records the source and verification evidence.

Appendix A. Exact integral schedule

Let $\ell = 30000000000000\,m$, for an integer $m > 0$. The output and source digit counts are:

Digit	Output count divided by m	Source count divided by m
0	24361752111000	27034502782000
1	4760008248000	2672750671000
2	878239641000	292746547000

They give the source degree and bank threshold

$$h_0 = 3258243765000\,m, \qquad T_0 = 3130753934820264\,m.$$

Every output word has degree $2h_0$ and cost $T_0 + D_0$, where $D_0 = 206565920180736\,m$. At prefix length s, the corresponding quantities are multiplied by s; the inverse-tree budget is $sD_0 + 1024$. The identities

$$\pi_1\ell = 2(\alpha_1 - \alpha_2)\ell, \qquad \pi_2\ell = 3\alpha_2\ell$$

give $C(Q) = 1024sh_0 + 3s\alpha_2\ell$. These equalities are proved over natural numbers in `Schedule.lean`; no rounding of the type counts is required.

For direct numerical reproduction, Table 2 gives the rational enclosures used in Proposition 4.1. The entries specify $10^{-18}l_X \le X \le 10^{-18}u_X$. Cross multiplication with these integers verifies every principal comparison.

Table 2. Exact natural-log rate enclosures, scaled by 10^{18}.

X	l_X	u_X
H	564524129743208140	564524129743208144
A	354403555221115172	354403555221115177
G	142141038618045131	142141038618045132
B	542022031540465943	542022031540466364
Q_*	48568421114506253	48568421114506283

Appendix B. Guide to the formal proof

All paths below are relative to `Submission/Weighted/`. The prefix `Global/` contains the adapted whole-solver callers.

Argument	Main source files
Euler identity and base coverage	`Cache.lean`
Weighted state counts and full query tree	`NearBankEnumeration.lean`, `FrozenQueryCost.lean`
Concrete kernel and its own finite costs	`KernelProgram.lean`, `KernelCosts.lean`
Prepared query and source/count semantics	`PreparedBudget.lean`, `SourceCountExecutionFactory.lean`
Exact logarithms and principal ledger	`Parameters.lean`, `GlobalParameters.lean`
Actual prefix budgets and global time	`Global/CountFrontendBudget.lean`, `Global/TernaryActualGlobalQueryRuntime.lean`
Simultaneous parameters and complete budget	`Global/TernaryConcreteProfile.lean`, `Global/TernaryProfileRuntime.lean`
Original-input execution and final theorem	`Global/TernaryColoredExecution.lean`, `Global/TernaryFinalSolver.lean`

References

1. Josh Alman and Virginia Vassilevska Williams, *Truly subquadratic 3SUM and truly subcubic APSP via triangles in sparse lopsided graphs*, 2026, `https://arxiv.org/abs/2610.06783`.
2. Big O Speedrun, *3SUM: problem specification and verified bound history*, 2026, Accessed October 31, 2026. `https://bigospeedrun.com/3sum`.
3. Andreas Björklund, Thore Husfeldt, Petteri Kaski, and Mikko Koivisto, *Fourier meets Möbius: Fast subset convolution*, Proceedings of the 39th Annual ACM Symposium on Theory of Computing, 2007, pp. 67–74.
4. c0ldzy17, Joshua Wharton, and Hrishik Sai Bojnal, *3sum-bound: ternary 3SUM executable and refined exponent certificates*, Lean source archive, with contributors, 2026, Immutable snapshot `8af82141280754b0c46d3dacb926426265da6f99`. `https://github.com/proof-factory/3sum-bound/tree/8af82141280754b0c46d3dacb926426265da6f99`.
5. Leonardo de Moura and Sebastian Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction – CADE 28, Lecture Notes in Computer Science, vol. 12699, Springer, 2021, pp. 625–635.
6. Swapnil Jain, *threesum-derand: deterministic 3SUM and weighted zero testing*, Research notes and source code, 2026, `https://github.com/Swapnil-jain/threesum-derand`.
7. The mathlib Community, *The Lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, 2020, pp. 367–381.

University of Science and Technology of China
Email address: `wangjh1@ustc.edu.cn`